

Lab 5: Claude Code CI Review

Alignment: Domains 3 and 4 - Claude Code Workflows and Structured Output Estimated time: 75 minutes Mode: Local simulation; no API key required

Objective

Create a deterministic CI review contract with JSON Schema, per-file review, integration review, false-positive criteria, and a severity-based exit gate.

Scenario

A pull-request pipeline needs actionable machine-readable findings. A single prose review is too difficult to parse and may miss cross-file behaviour.

Prerequisites

- Python 3.10 or newer.
- A terminal opened in this lab folder.
- No production account, network access, or secret is required.

Architecture before execution

Read the source and identify the control boundary, the evidence artifact, and the terminal condition before running it. All side effects are confined to this lab's evidence/ folder.

Procedure

```
1. Inspect the finding schema and identify every required field.`bashpython3 -m
json.tool finding.schema.json`### 2. Run the offline deterministic review
fixture.`bashpython3 -B ci_review.py fixtures/changes.json`### 3. Inspect the review artifact
and distinguish per-file findings from integration findings.`bashpython3 -m json.tool
evidence/review.json`### 4. Review the documented claude -p command and map its
output contract to the same schema.`bashsed -n '1,220p' claude-command.txt`### 5. Run
the verifier and save its output.`bashmkdir -p evidence && python3 -B verify.py | tee
evidence/verification.txt`
```

Acceptance checks

- [] The schema requires file, line, severity, evidence and remediation.
- [] Per-file and integration passes are separate.
- [] Style-only guesses are filtered unless a stated rule is violated.
- [] The exit gate fails only on high-severity findings.

Reflection

What defect can an integration pass detect that two independent per-file reviews cannot?

Evidence and completion

Save terminal output in evidence/verification.txt. The lab is complete when the verifier prints PASS, every acceptance check is satisfied, and your reflection identifies one design trade-off.

Troubleshooting

- Run commands from this lab folder so relative paths resolve correctly.
- Use python3 -B to prevent learner machines from creating __pycache__ artifacts.
- If a check fails, read the named failed assertion, inspect the referenced file, and rerun only after correcting the underlying design.

Clean-up

Remove only files you created outside this folder. Never store API keys, access tokens, customer data, or production logs in lab evidence.